

电子与信息学报
Journal of Electronics & Information Technology
ISSN 1009-5896, CN 11-4494/TN

《电子与信息学报》网络首发论文

题目: 融合代码序列与属性图的源代码漏洞检测方法
作者: 杨宏宇, 罗靖川, 成翔, 胡俊成
收稿日期: 2025-05-27
网络首发日期: 2025-11-05
引用格式: 杨宏宇, 罗靖川, 成翔, 胡俊成. 融合代码序列与属性图的源代码漏洞检测方法[J/OL]. 电子与信息学报.
<https://link.cnki.net/urlid/11.4494.TN.20251105.1009.002>



网络首发: 在编辑部工作流程中, 稿件从录用到出版要经历录用定稿、排版定稿、整期汇编定稿等阶段。录用定稿指内容已经确定, 且通过同行评议、主编终审同意刊用的稿件。排版定稿指录用定稿按照期刊特定版式(包括网络呈现版式)排版后的稿件, 可暂不确定出版年、卷、期和页码。整期汇编定稿指出版年、卷、期、页码均已确定的印刷或数字出版的整期汇编稿件。录用定稿网络首发稿件内容必须符合《出版管理条例》和《期刊出版管理规定》的有关规定; 学术研究成果具有创新性、科学性和先进性, 符合编辑部对刊文的录用要求, 不存在学术不端行为及其他侵权行为; 稿件内容应基本符合国家有关书刊编辑、出版的技术标准, 正确使用和统一规范语言文字、符号、数字、外文字母、法定计量单位及地图标注等。为确保录用定稿网络首发的严肃性, 录用定稿一经发布, 不得修改论文题目、作者、机构名称和学术内容, 只可基于编辑规范进行少量文字的修改。

出版确认: 纸质期刊编辑部通过与《中国学术期刊(光盘版)》电子杂志社有限公司签约, 在《中国学术期刊(网络版)》出版传播平台上创办与纸质期刊内容一致的网络版, 以单篇或整期出版形式, 在印刷出版之前刊发论文的录用定稿、排版定稿、整期汇编定稿。因为《中国学术期刊(网络版)》是国家新闻出版广电总局批准的网络连续型出版物(ISSN 2096-4188, CN 11-6037/Z), 所以签约期刊的网络版上网络首发论文视为正式出版。

融合代码序列与属性图的源代码漏洞检测方法

杨宏宇^{*①②} 罗靖川^① 成翔^③ 胡俊成^④^①(中国民航大学安全科学与工程学院 天津 300300)^②(中国民航大学计算机科学与技术学院 天津 300300)^③(扬州大学信息工程学院 扬州 225127)^④(吉林大学计算机科学与技术学院 长春 130012)

摘要：针对现有源代码漏洞检测方法无法充分提取并有效融合代码特征，导致检测模型学习不全面、漏洞检测性能不佳等问题，该文提出一种融合代码序列与属性图的源代码漏洞检测方法。首先，获取代码序列特征表示和代码属性图特征表示。其次，使用预训练模型UniXcoder提取代码语义特征并使用残差门控图卷积网络提取代码图结构特征。然后，构建一种用于特征融合的多模态注意力融合网络，通过学习代码的语义特征与图结构特征之间的交互关系，形成对漏洞检测任务更有价值的融合特征。最后，通过引入插值预测分类器并调整模型关注点，提升模型针对不同特性样本的泛化性并优化漏洞检测性能。多个数据集上的实验结果表明，所提方法具有良好检测效果，准确率可提升0.08%~1.38%，精确率可提升5.19%~8.15%。

关键词：漏洞检测；代码属性图；预训练模型；插值预测

中图分类号：TP393

文献标识码：A

文章编号：1009-5896(2025)12-0001-11

DOI: 10.11999/JEIT250470

CSTR: 32379.14.JEIT250470

1 引言

随着信息技术的发展，软件在社会生活的各个领域扮演着关键角色。然而，软件漏洞的存在为网络攻击提供了可乘之机。近几年，国际权威漏洞数据库通用漏洞披露(Common Vulnerabilities and Exposures, CVE)和美国国家漏洞数据库(National Vulnerability Database, NVD)中披露的软件漏洞数量逐年上升，尤其是2022年披露的漏洞数量是前几年的数倍^[1]。

作为一种典型的白盒漏洞检测技术，源代码漏洞检测方法能及时、准确地检测漏洞，可以显著提升软件系统的安全性和可靠性。然而，传统的基于机器学习的源代码漏洞检测方法存在过度依赖专家手动提取特征、泛化能力不足等问题，导致漏洞检测性能低下，不利于漏洞检测。近年来，深度学习技术在源代码漏洞检测领域获得广泛关注，相关研究持续增长。尽管如此，现有基于深度学习的方法多采用单一网络结构，从单一视角提取代码特征，

限制模型对多维度代码特征的全面建模能力。部分研究尝试从不同维度提取代码特征以增强特征表示能力，但所采用的特征融合策略较为简单，通常以特征拼接或加权组合方式进行处理，忽视了不同维度特征间的关联性，导致特征融合效果受限。此外，现有方法普遍在漏洞检测阶段面临泛化能力不足的问题，无法根据目标数据集的差异性合理调整检测模型关注点，进而限制模型对关键漏洞特征的学习能力，影响实际应用中的检测性能。

为解决以上问题，本文提出一种融合代码序列与属性图的源代码漏洞检测方法(source code vulnerability detection method integrating Code Sequences and Property Graphs, CSPG)。该方法通过学习代码序列中的语义信息和代码属性图中的图结构信息，充分提取代码特征并构造多模态注意力融合网络进行特征融合，通过引入插值预测分类器优化检测，有效提升漏洞检测性能。本文主要贡献如下：

(1)设计一种多模态注意力融合网络(Multimodal Attention Fusion Network, MAFN)作为特征融合网络。该网络能对预训练模型提取出的代码语义特征和图神经网络提取出的图结构特征进行有效融合，使得模型在融合特征时动态聚焦于关键信息，生成有助于提升漏洞检测模型灵敏度的融合特征，进而增强检测效果。

(2)构造一种插值预测分类器。基于线性插值的思想，提出插值预测分类器。通过控制多个预测

收稿日期：2025-05-27；改回日期：2025-10-01

*通信作者：杨宏宇 yhyxlx@hotmail.com

基金项目：国家自然科学基金民航联合研究基金重点项目(U2433205)，江苏省基础研究计划自然科学基金青年基金项目(BK20230558)

Foundation Items: The Civil Aviation Joint Research Fund Project of the National Natural Science Foundation of China (U2433205), The Jiangsu Provincial Basic Research Program Natural Science Foundation - Youth Fund Project (BK20230558)

结果间的权衡,能针对不同特性的代码样本合理调整漏洞检测模型的关注点,使检测模型更加聚焦于特性样本的关键特征上,从而提升漏洞检测性能。同时,插值预测分类器能增强模型对不同特性样本的适应能力,有效提高检测模型的泛化能力。

(3)提出一种融合预训练模型与图神经网络的源代码漏洞检测模型。由预训练模型学习代码序列中的语义信息,由图神经网络学习代码属性图中的图结构信息,检测模型能够充分获取代码的语义特征和图结构特征。通过在多个数据集上的实验对比,结果表明该模型具有良好的漏洞检测效果,在检测准确率、精确率和F1分数上均有提升。

2 相关工作

近年来,随着深度学习技术的发展,越来越多的研究利用深度学习方法进行源代码漏洞检测。与传统机器学习漏洞检测方法相比,基于深度学习的漏洞检测方法利用深度神经网络自动提取和学习源代码中的隐含漏洞语义特征,可以有效减少对专家手动定义特征的依赖,并能提高特征提取的效率和准确性。基于深度学习的漏洞检测技术在安全领域逐渐获得广泛认可,成为当前研究的热点。目前,基于深度学习的源代码漏洞检测方法主要包括基于代码序列特征的方法和基于图结构特征的方法。

基于序列特征的方法^[2-7]是指将预处理后的代码矢量化为序列标记,以生成代码序列特征的向量表示,检测模型通过学习代码序列特征进行漏洞检测。有研究者利用CodeBERT^[8]模型进行源代码漏洞检测,将代码函数片段作为模型输入。凭借CoderBERT丰富的预训练和强大的迁移学习能力,该方法能够捕获代码中的语法和语义特征,可有效识别复杂代码模式和潜在漏洞。虽然CodeBERT能捕获代码的局部语义,但对全局代码的逻辑关系和上下文理解能力较弱。LineVul^[2]方法是一种基于Transformer的细粒度漏洞检测方法,它能够进行行级漏洞检测,这比文件或函数级别的检测更为精细。LineVul利用BERT架构和自注意力机制获取代码中的长期依赖关系,但该方法仅基于局部上下文的检测,无法覆盖全局上下文。上述基于序列特征的方法缺少对代码结构的理解,捕获全局上下文信息的能力不足。

基于图结构特征的方法^[9-17]通过图结构来表示程序代码特征,并利用图神经网络学习图结构信息。在这些图中,节点表示程序元素,边表示它们之间的关系。Zhou等人^[9]提出Devign方法,通过构造一种基于抽象语法树、控制流图、程序依赖图和

自然代码序列组合的复合图,Devign能够高效理解代码结构,但该方法对代码的语义理解有限,在处理需要复杂语义分析的漏洞时表现不佳。Reveal^[10]方法利用门控图神经网络对代码属性图进行分析,能够充分理解代码的控制流、数据流等结构信息,但该方法对代码语义信息的捕获能力较弱。SCALE^[11]方法中利用抽象语法树和注释信息,构造出结构化自然语言注释树进行漏洞检测。尽管该方法借助结构化自然语言注释树来提升检测模型对代码语义特征的学习能力,但注释信息本身难以全面涵盖代码语义特征,导致模型对语义信息的理解存在局限性。此外,由于该方法中的注释信息是借助大语言模型生成的,使得整个方法的计算开销较大。Vul-LMGNNs^[12]方法分别使用门控图神经网络提取代码图结构特征,使用CodeBERT提取代码语义特征,使模型能同时学习到代码图结构特征和语义特征,进一步提升检测性能。但Vul-LMGNNs中特征融合不充分,仅通过简单拼接多个特征作为特征融合方法,严重忽视特征间潜在的关联性。上述基于图结构特征的方法侧重于学习代码结构特征,未能高效提取并融合代码语义信息。

为解决以上问题,提升漏洞检测性能,本文提出CSPG方法。与基于序列特征的方法相比,CSPG借助图神经网络对代码属性图进行建模,能有效提升模型对图结构信息的理解。与基于图结构特征的方法相比,CSPG引入预训练模型以获取更丰富的代码语义信息,增强模型对代码语义特征的学习能力。此外,CSPG进一步优化特征融合方法,加强模型对特征间交互关系的理解,从而实现对代码特征的充分学习。

3 方法设计

3.1 总体框架

本文提出的CSPG由特征表示、特征提取、特征融合和漏洞检测4部分组成(如图1所示)。每个部分的核心功能如下:

(1)特征表示。获取代码序列和代码属性图(Code Property Graph, CPG)的向量表示,利用词嵌入和节点嵌入技术,分别将代码序列和图节点映射为固定维度的向量,用于后续的特征提取。

(2)特征提取。使用预训练模型UniXcoder^[18]捕获代码上下文信息,提取代码语义特征。使用残差门控图卷积网络(Residual Gated Graph Convolution Network, RGGCN)^[19]获取CPG中复杂的结构信息,提取图结构特征。

(3)特征融合。基于注意力机制构造MAFN作

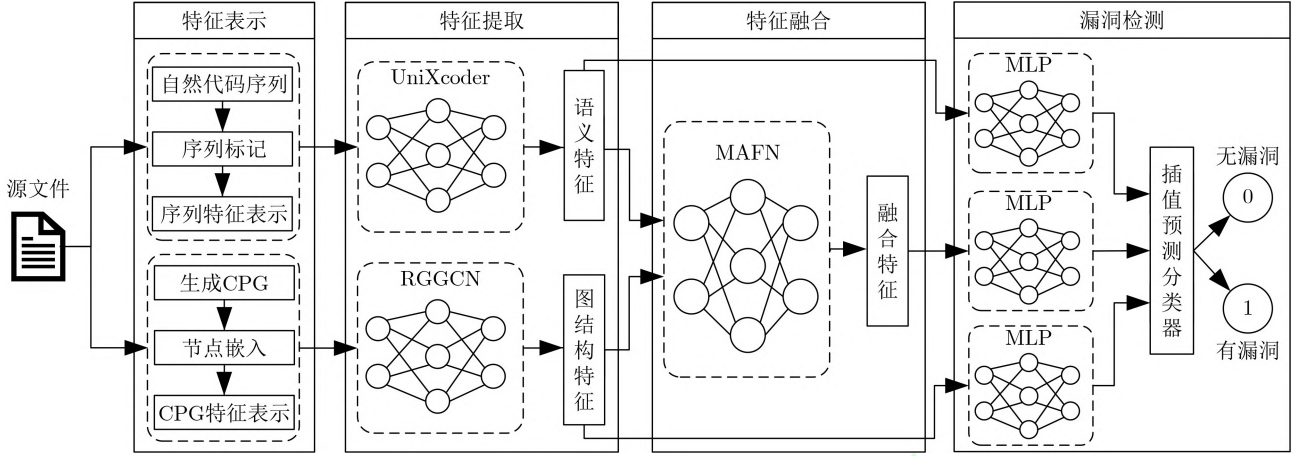


图1 CSPG框架图

为特征融合网络，MAFN能够通过学习代码的语义特征与图结构特征之间的交互关系，形成对漏洞检测任务更有价值的融合特征。

(4)漏洞检测。利用多层感知机(Multilayer Perceptron, MLP)分别对语义特征、图结构特征和融合特征进行分类预测，使用插值预测分类器优化漏洞检测，得到最终检测结果。

3.2 特征表示

3.2.1 代码序列特征表示

从源代码到生成代码的序列特征表示有两个步骤，先获取自然代码序列，然后生成对应的序列标记，便可得到代码序列特征表示。

采用文献[9]中提出的方法，通过转换函数级代码来获取自然代码序列。该方法根据源代码的自然顺序得到自然代码序列，从而保留代码的逻辑顺序。设源代码文本为 $C=\{c_1, c_2, \dots, c_n\}$ ，其中 c_i 表示代码中的第 i 个元素(如标识符、关键字等)， n 为代码序列长度。则自然代码序列 S 可形式化表示为

$$S = \text{Extract}(C) \quad (1)$$

其中， $\text{Extract}(\cdot)$ 为序列提取操作，能保留源代码执行顺序中的语法元素排列。

然后再利用预训练模型UniXcoder中的Tokenizer对自然代码序列 S 进行分词，表示为

$$T = \text{Tokenizer}(S) = (t_1, t_2, \dots, t_m) \quad (2)$$

其中， t_i 表示生成的第 i 个序列标记， m 为标记数量。

最后将序列标记 T 进行编码，则可获得代码序列特征的向量表示 E ，记编码函数为Encoder $(\cdot)$ ，则

$$E = \text{Encoder}(T) = (e_1, e_2, \dots, e_m) \quad (3)$$

其中， $e_i \in \mathbb{R}^d$ 表示第 i 个序列标记对应的 d 维特征向

量，特征向量 $E \in \mathbb{R}^{m \times d}$ 表示自然代码序列在嵌入空间中的分布。

3.2.2 代码属性图特征表示

得到代码属性图的特征表示需要生成CPG和节点嵌入两个环节。首先使用开源代码分析工具Joern对数据集中的每一条数据所包含的函数代码段生成对应的CPG。然后利用CodeBERT对CPG进行节点嵌入。完成节点嵌入后，便可得到代码属性图特征的向量表示。

CPG是一种将抽象语法树(Abstract Syntax Tree, AST)、控制流图(Control Flow Graph, CFG)和程序依赖图(Program Dependence Graph, PDG)融合的综合图结构，能够捕获更多维度的信息，并被证明对漏洞检测非常有效。CPG中的每个节点都包含Code和Type属性，Code表示节点中包含的源代码，Type表示节点的类型。

利用CodeBERT进行CPG节点嵌入，可得到图 $g=(V, H, A)$ ，其中 V 表示节点集合、 H 表示特征矩阵、 A 表示邻接关系。CodeBERT能够将节点的语义信息转换为高维向量表示，使得模型能够更好地理解不同节点在语义上的相似性和差异，利于特征提取。

3.3 特征提取

3.3.1 代码语义特征提取

为能有效从代码序列中提取出代码语义特征，便采用预训练模型UniXcoder进行代码语义特征提取。UniXcoder的核心架构基于Transformer，共包含12层双向Transformer，支持对代码和自然语言的混合建模。采用自注意力机制，使其可以捕获代码中的长距离依赖关系。此外，通过大量预训练，UniXcoder能有效克服数据集中漏洞样本与非漏洞样本的不平衡所带来的过拟合问题，利于漏洞检测。通过微调预训练模型，UniXcoder可充分利用

上下文信息来学习每个代码片段的语义特征 F_1 ，其形式化表示为

$$F_1 = \text{UniXcoder}(\{e_1, e_2, \dots, e_m\}) \quad (4)$$

将代码序列特征的向量表示 E 输入至UniXcoder的Transformer编码器中，通过堆叠Transformer编码层，可获得代码语义信息。其中，代码语义信息包含多个隐藏状态，选取最后一层中第1个token的隐藏状态表示代码片段的语义特征 F_1 。

3.3.2 代码图结构特征提取

选择使用RGGCN进行代码图结构特征提取，RGGCN作为一种结合了门控机制和残差学习的图卷积网络模型，旨在提高图结构数据的特征提取能力，尤其在处理复杂图结构时有出色表现。CPG是由AST、CFG和PDG组合而成的综合图，RGGCN通过多层次的图卷积，能够深入获取CPG中复杂的结构信息，有助于CPG特征提取。

将CPG的节点嵌入后，得到图 $g=(V, H, A)$ ，其中 V 表示节点集合、 H 表示特征矩阵、 A 表示邻接关系。每一个节点 $v \in V$ 都具有一个初始特征向量 $h^{(0)}$ ，所有特征向量组合构成整个图的特征矩阵 H 。首先利用邻接矩阵 A 对节点的特征进行聚合，生成聚合特征矩阵 H' ，计算公式为

$$H' = \sigma(\hat{A}HW) \quad (5)$$

其中， $\hat{A}$ 是归一化后的邻接矩阵， H 表示当前层的节点特征矩阵， W 是权重矩阵， σ 为激活函数。

然后计算门控值 G ，门控机制可以决定从邻接节点聚合来的信息与节点自身特征的融合程度，其具体计算为

$$G = \sigma(HW_g + b_g) \quad (6)$$

其中，门控值 $G \in (0, 1)$ ， W_g 和 b_g 是门控机制的可学习参数。

然后，利用门控值更新节点特征，计算公式为

$$H_{\text{gate}} = G \odot H' + (1 - G) \odot H \quad (7)$$

其中， H_{gate} 为经过门控机制选择性保留后的特征矩阵， $\odot$ 表示逐元素相乘。

为缓解梯度消失问题并保留初始特征，将残差连接引入到每层

$$H^{(l+1)} = H_{\text{gate}} + H^{(l)} \quad (8)$$

其中， $H^{(l)}$ 是上一层的输出特征。

最后，将多层RGGCN堆叠，便可得到最终输出节点的高层语义特征，即提取出代码图结构特征 F_2 。

3.4 特征融合

为能充分融合代码的语义特征和图结构特征，提升特征间的关联性，基于注意力机制构造出MAFN作为特征融合网络。

MAFN是一种用于结合来自不同模态的特征信息的特征融合方法，能够让模型在融合特征时动态聚焦于重要的信息。在漏洞检测中，MAFN通过学习代码的语义特征与图结构特征之间的交互关系，形成对漏洞检测任务更有价值的融合特征，MAFN的结构如图2所示。

图2中， F_1 为UniXcoder提取的代码语义特征， F_2 为RGGCN提取的图结构特征。先将 F_1 映射到 Q 上， F_2 映射到 K 和 V 上，计算公式为

$$\left. \begin{aligned} Q &= F_1 \times W_Q \\ K &= F_2 \times W_K \\ V &= F_2 \times W_V \end{aligned} \right\} \quad (9)$$

计算 Q 和 K 的相似性注意力得分，得到 F_2 重新加权后的特征表示 F'_2 ，计算公式为

$$F'_2 = \text{Softmax}(QK^T d_k^{-1/2})V \quad (10)$$

其中， d_k 是 K 的维度大小，用于计算注意力分数时的缩放因子。

借助语义信息，引导漏洞检测模型聚焦于代码

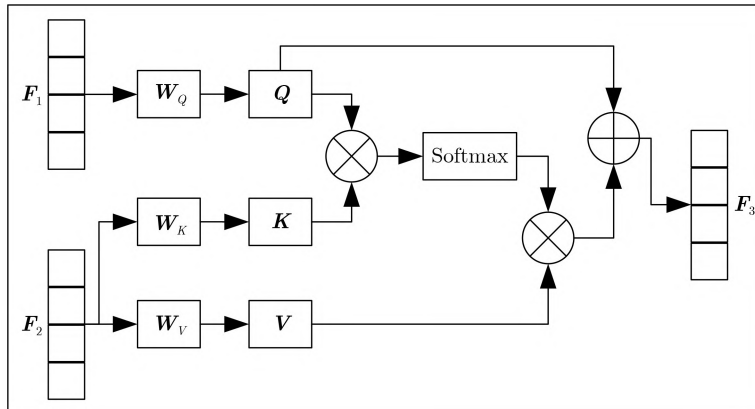


图2 MAFN结构图

结构内潜在漏洞模式。其中， F_2 中最重要的部分将被赋予更高的权重，即重点关注对漏洞检测有益的结构信息。然后再与 F_1 在 Q 上的映射进行拼接，得到最终的融合特征 F_3 。

$$F_3 = \text{Concat}(Q, F_2') \quad (11)$$

将 Q 与 F_2' 进行拼接，有效保留代码的语义信息，确保模型对代码特征的全面获取。

上述特征融合网络的设计能够帮助漏洞检测模型聚焦代码中潜在的漏洞位置。例如，某个函数调用的语义特征和其控制流图的依赖结构可能共同指示出潜在的漏洞，MAFN能够动态权衡这两者。同时，MAFN能充分获取代码语义特征与代码图结构特征之间的依赖关系，自动为不同模式特征分配权重，能够自适应地融合信息，有助于模型理解复杂的漏洞模式，可以有效提升漏洞检测性能。

3.5 漏洞检测

3.5.1 漏洞检测过程

为解决漏洞检测模型在漏洞检测阶段泛化性不足的问题，基于BertGCN^[20]提出的线性插值思想，构造插值预测分类器。针对不同特性的样本数据，插值预测分类器通过合理调节参数值，进而调整漏洞检测模型的关注点，能有效使模型侧重关注特性样本中的关键代码特征，最终得到检测结果。该机制能增强模型对不同特性样本的适应能力，有效提高检测模型的泛化能力。漏洞检测网络架构如图3所示。

首先，使用MLP对此前生成的代码的语义特征 F_1 、图结构特征 F_2 和融合特征 F_3 进行二分类预测，分别得到语义特征预测结果 P_1 、图结构特征预测结果 P_2 和融合特征预测结果 P_3 ，计算公式为

$$\begin{cases} P_1 = \text{MLP}(F_1) \\ P_2 = \text{MLP}(F_2) \\ P_3 = \text{MLP}(F_3) \end{cases} \quad (12)$$

其中， P_1 、 P_2 和 P_3 均是形状为 $(B, 2)$ 的向量，“ B ”表示批量大小，“2”表示列数，实验中的批量大小设置为“32”。向量中每一行即代表对一条样本数据的检测，其中第1列是预测为“0”的概率，即“无漏洞”，第2列是预测为“1”的概率，即“有漏洞”。

然后，构造插值预测分类器，控制上述3个预测结果对最终生成的预测结果 P 的重要程度，插值预测分类器的计算公式为

$$P = P_1 \times \lambda_1 + P_2 \times \lambda_2 + P_3 \times (1 - \lambda_1 - \lambda_2) \quad (13)$$

其中， λ_1 和 λ_2 满足关系

$$\begin{cases} 0 \leq \lambda_1 \leq 1 \\ 0 \leq \lambda_2 \leq 1 \\ 0 \leq \lambda_1 + \lambda_2 \leq 1 \end{cases} \quad (14)$$

由式(13)得到的最终预测结果 P 为一个形状为 $(B, 2)$ 的向量，其中 $B=32$ 。

最后，对预测结果 P 按行进行处理，1行即代表对1条样本数据的检测。比较该行中第1列和第2列的概率值大小，选取概率较大的一列作为检测结果，其中第1列表示“无漏洞”，第2列表示“有漏洞”。综上处理，即可得到1个批量大小样本的漏洞检测结果。

3.5.2 参数设置讨论

插值预测分类器通过调节 λ_1 和 λ_2 的数值，能够控制 P_1 、 P_2 和 P_3 预测结果之间的权衡，从而调整漏洞检测模型的关注点，使模型重点关注特性样本中的关键代码特征，能有效优化检测结果。

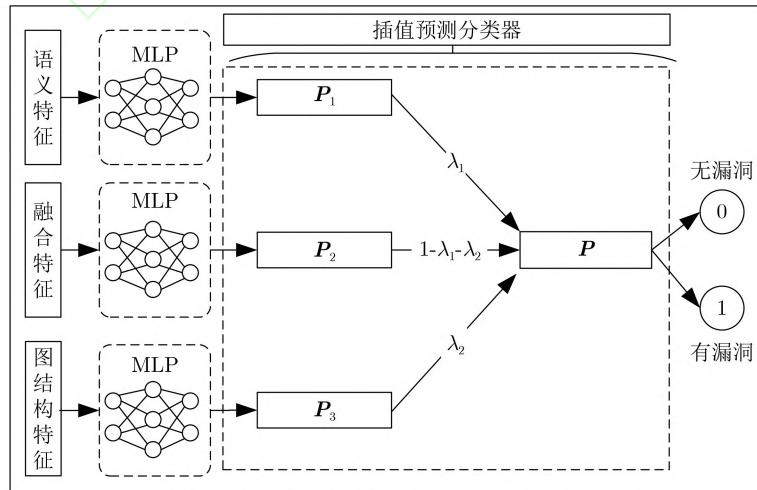


图3 漏洞检测网络架构

λ_1 和 λ_2 取值的不同,对检测结果有着不同影响。当 $\lambda_1=1, \lambda_2=0$ 时,表示完全使用UniXcoder模型;当 $\lambda_1=0, \lambda_2=1$ 时,表示完全使用RGGCN模型;当 $\lambda_1 \neq 1, \lambda_2 \neq 1$ 时,可以根据需求合理调整 P_1, P_2 和 P_3 预测结果,如针对代码属性图较为复杂的学习样本,可适当加大 λ_2 的值,并减小 λ_1 的值。调整参数值后,漏洞检测模型将更加关注图结构的学习,能从代码属性图中更深层次地提取出图结构信息,可以有效提升漏洞检测性能。

因此,针对不同特性的样本数据检测时,插值预测分类器通过合理调节参数值,能够控制漏洞检测模型关注特性样本所对应的特征属性,加强模型对该类特征的学习,进而提升漏洞检测性能,增强模型检测的泛化能力。

4 实验及结果分析

4.1 实验设置

为验证所提方法有效性,选取来源于真实软件项目的Devign数据集^[9]、Reveal数据集^[10]和基于开源项目构造的SVulD数据集^[21]作为实验数据。

Devign数据集是为源代码漏洞检测任务而设计的公开数据集之一,旨在支持基于图神经网络等深度学习方法的漏洞检测研究。Devign数据集主要收集自广泛使用的开源C语言代码库,代码片段大部分来源于实际的开源项目。作为一个公开且通用的数据集,Devign数据集已成为评估不同漏洞检测算法的标准,便于算法性能的横向对比。

Reveal数据集是为漏洞检测任务设计的高质量数据集之一,专注于帮助研究者识别和检测代码中的安全漏洞。Reveal数据集主要来源于Linux Debian Kernel和Chromium项目的代码历史版本,通过对项目提交记录、补丁和漏洞报告进行分析提取。

SVulD数据集是一个合成的代码漏洞数据集,主要针对不同类型的常见漏洞进行人工构造和标注。相比于从真实项目中收集的代码数据集,SVulD具有多样的代码结构,以及多样化的代码模式,能够更好地模拟实际开发环境中的多样性。同时SVulD数据集中所有代码样本均带有明确的标签,标注漏洞的具体类型,适合监督学习模型的训练和验证。

在实验中,使用的GPU型号为NVIDIA GeForce RTX 4090 24 GB,内存大小为90 GB。NVIDIA驱动程序版本为550.78, CUDA版本为12.4。软件环境配置Python版本为3.10.12, Pytorch版本为2.4.1, CPG生成工具Joern版本为v1.1.1700。实验

采用准确率、精确率、召回率和F1分数作为评价指标。

4.2 实验结果与分析

4.2.1 与现有方法的检测性能对比

为验证本文方法CSPG的有效性,选取Devign^[9], CodeBERT^[8], Reveal^[10], LineVul^[2], Vul-LMGNNs^[12], SCALE^[11]作为基线方法,分别在Devign数据集、Reveal数据集和SVulD数据集上,将CSPG与这些漏洞检测方法进行性能对比。不同检测方法的漏洞检测结果如表1、表2和表3所示。

由表1–表3可知,在Devign数据集上,CSPG的准确率较SCALE提升1.38%,精确率较CodeBERT提升5.19%。在Reveal数据集上,CSPG的准确率较SCALE提升0.08%,精确率与SCALE的52.32%最为接近。在SVulD数据集上,CSPG的准

表 1 Devign数据集上的检测性能对比(%)

检测方法	准确率	精确率	召回率	F1分数
Devign	55.02	52.90	56.37	54.58
CodeBERT	63.49	64.46	43.37	51.85
Reveal	62.38	59.80	53.71	56.59
LineVul	62.37	61.55	48.21	54.07
Vul-LMGNNs	65.19	63.64	53.85	58.33
SCALE	66.18	61.88	68.69	65.11
CSPG	67.56	69.65	50.31	58.42

表 2 Reveal数据集上的检测性能对比(%)

检测方法	准确率	精确率	召回率	F1分数
Devign	87.49	36.65	31.55	33.91
CodeBERT	88.58	47.80	41.04	44.16
Reveal	85.37	29.90	40.91	33.87
LineVul	88.87	48.60	56.15	49.10
Vul-LMGNNs	89.16	44.38	33.65	38.27
SCALE	90.02	52.32	78.69	62.85
CSPG	90.10	50.82	43.87	47.09

表 3 SVulD数据集上的检测性能对比(%)

检测方法	准确率	精确率	召回率	F1分数
Devign	73.57	9.72	50.31	16.29
CodeBERT	80.56	14.33	55.32	22.76
Reveal	82.58	12.92	40.08	19.31
LineVul	80.57	15.95	64.45	25.58
Vul-LMGNNs	86.20	57.83	45.39	50.86
SCALE	87.63	22.56	57.03	32.33
CSPG	87.76	65.98	45.50	53.86

准确率较SCALE方法提升0.13%，精确率较Vul-LMGNNs提升8.15%。总体而言，CSPG有较高的准确率和精确率，这是由于该方法有效结合了UniXcoder提取的代码语义信息和RGGCN提取的图结构信息。虽然CodeBERT也可以很好地获取代码语义，但它缺乏对复杂代码样本结构的理解，故检测性能较低。而Devign采用门控图神经网络来获取代码图结构信息，但其缺少对代码序列所包含的语义信息的理解，故表现不佳。而CSPG通过充分融合代码语义特征和图结构特征提升漏洞检测性能，所以在准确率和精确率方面有较为出色的性能表现。

4.2.2 特征融合的有效性验证

为验证语义与图结构特征结合的有效性，分别在Devign, Reveal和SVuID数据集上进行消融实验。在相同实验环境下，将CSPG与UniXcoder, RGGCN进行性能对比。其中，UniXcoder以数据集中样本的代码序列特征表示作为模型输入，RGGCN以数据集中样本的代码属性图特征表示作为模型输入，经过模型处理，得到对应实验结果，实验结果如图4所示。

由图4可见，与UniXcoder和RGGCN相比，CSPG在准确率、精确率和F1分数指标上均有一定程度的提升。上述结果表明，将语义特征与图结构特征相结合，对于提升漏洞检测任务的性能具有重要作用；通过深度融合代码的语义信息与结构信息，CSPG能够从多维度挖掘漏洞样本的潜在特征，实现对恶性样本的更精准识别。这种特征融合策略使得CSPG在漏洞检测任务中展现出更强的适应性和鲁棒性。

通过进一步分析可知，UniXcoder在漏洞检测性能上显著优于RGGCN。说明：

(1)预训练模型凭借对复杂代码模式和上下文信息的捕获能力，能够从代码中提取更丰富、更细

致的语义特征，在漏洞检测任务中具有显著的效果。依托大规模预训练语料和迁移学习能力，UniXcoder对代码上下文及语义关系的建模能力得到了充分发挥，使其在检测漏洞时更具优势。

(2)RGGCN主要依赖图神经网络对代码的图结构特征进行建模，其在捕获结构化信息上有所表现，但在应对需要语义背景的漏洞样本时，检测能力相对不足。

实验结果表明，CSPG展现出的这种多维度融合特性，不仅可以提升漏洞检测的精度，还能在处理复杂的代码场景提供更加全面、高效的解决方案。

4.2.3 插值预测机制的有效性验证

为验证插值预测机制的有效性，在Devign数据集上，通过调整不同的 λ_1 和 λ_2 参数值进行实验对比。

实验中， λ_1 和 λ_2 的取值均为[0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]，并按照式(14)进行参数组合实验，实验结果如表4所示。

表4中的行为 λ_1 的取值，列为 λ_2 的取值，表格内容为不同参数值组合下实验结果的指标数值，“*”表示没有该种组合情况。

由表4可知，模型的漏洞检测性能与参数 λ_1 和 λ_2 的设定密切相关。通过调整 λ_1 和 λ_2 ，可以显著优化模型的检测能力。当 λ_1 , λ_2 和 $1-\lambda_1-\lambda_2$ 均不为零时，即融合了语义特征预测结果 P_1 、图结构特征预测结果 P_2 和融合特征预测结果 P_3 三者的贡献时，模型性能达到最佳状态。这表明，在漏洞检测任务中，充分结合多种特征来源的预测结果，可以更全面地获取代码中的潜在漏洞信息。

由表4中的对比实验结果进一步揭示了不同特征在漏洞检测中的作用。当 $\lambda_1=0$ 时，即模型不使用UniXcoder的预测结果，检测性能明显低于 $\lambda_2=0$ 时，即不使用RGGCN的预测结果。这一现象

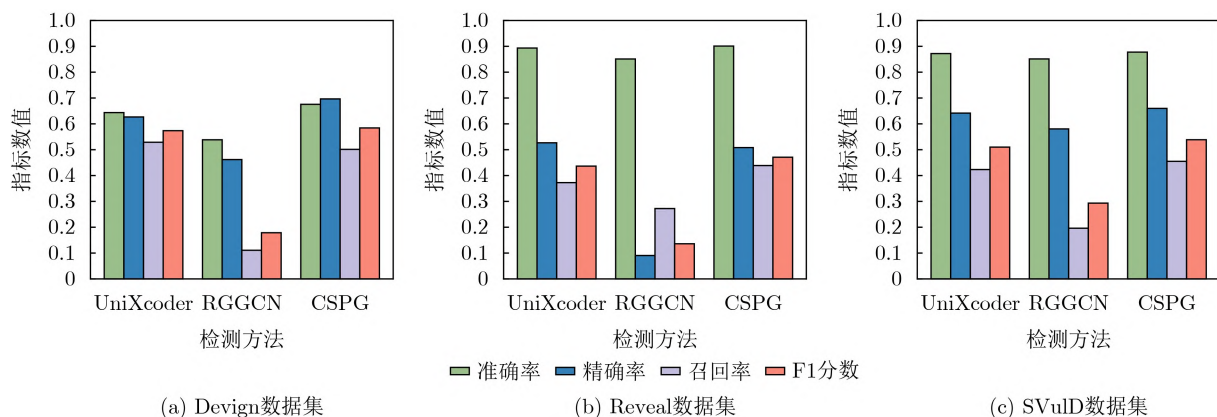


图4 不同数据集上的消融实验结果

表4 各参数值准确率(%)对比

	$\lambda_1=0$	$\lambda_1=0.1$	$\lambda_1=0.2$	$\lambda_1=0.3$	$\lambda_1=0.4$	$\lambda_1=0.5$	$\lambda_1=0.6$	$\lambda_1=0.7$	$\lambda_1=0.8$	$\lambda_1=0.9$	$\lambda_1=1.0$
$\lambda_2=0$	54.87	63.05	65.03	66.69	65.98	66.46	64.32	65.31	65.55	64.48	64.36
$\lambda_2=0.1$	54.19	65.66	63.61	62.90	66.53	67.56	64.87	64.04	63.73	66.57	*
$\lambda_2=0.2$	55.06	64.68	66.10	65.03	66.57	66.85	65.43	66.42	64.60	*	*
$\lambda_2=0.3$	55.06	63.45	65.07	64.72	64.95	65.74	64.48	65.78	*	*	*
$\lambda_2=0.4$	54.51	64.20	66.57	66.02	65.43	66.10	65.27	*	*	*	*
$\lambda_2=0.5$	54.63	61.19	65.55	63.21	65.55	64.68	*	*	*	*	*
$\lambda_2=0.6$	55.22	63.21	64.68	65.43	64.56	*	*	*	*	*	*
$\lambda_2=0.7$	54.15	63.96	64.75	64.87	*	*	*	*	*	*	*
$\lambda_2=0.8$	54.47	63.29	65.19	*	*	*	*	*	*	*	*
$\lambda_2=0.9$	53.56	63.21	*	*	*	*	*	*	*	*	*
$\lambda_2=1.0$	53.32	*	*	*	*	*	*	*	*	*	*

验证了4.2.2节的观点：与图神经网络相比，预训练模型在获取复杂代码模式和上下文表示方面更加出色，因而对提升漏洞检测性能更为有效。

综上，合理设置 λ_1 和 λ_2 的值对于模型性能至关重要。通过平衡各特征的贡献，插值预测机制能够充分发挥语义特征、图结构特征以及融合特征的优势，使模型在漏洞检测任务中达到最佳性能。这一结果能够充分验证插值预测机制在漏洞检测任务中的有效性和实用性。

4.2.4 预训练模型性能对比

为验证本方案结合不同预训练模型对漏洞检测性能的影响，在Devign数据集上分别选用CodeBERT^[8]、GraphCodeBERT^[22]、CodeBERT-CPP^[23]和LongCoder^[24]4种预训练模型替代CSPG中的UniXcoder进行测试，实验结果如图5所示。

由图5可见，在Devign数据集上的测试中，CSPG方法在准确率和精确率上表现最佳。CSPG的优越性能主要得益于其采用UniXcoder进行代码语义学习。UniXcoder通过多任务学习强化了对代码语义、

语法以及跨模态特征的深度理解，能够全面获取漏洞检测所需的特征信息。CodeBERT和GraphCodeBERT侧重单一任务的优化，未能充分展现对代码语义的广泛表示。UniXcoder在处理长代码序列时表现突出，能够有效获取全局依赖关系，从而在漏洞检测任务中具有更好的检测效果。尽管LongCoder针对长序列能进行特殊优化，但由于其更倾向于代码生成任务，导致在漏洞检测方面效果不及UniXcoder。

5 结束语

为解决目前源代码漏洞检测方法中特征融合不充分，以及漏洞检测阶段泛化性不足等问题，提出一种融合代码序列与属性图的源代码漏洞检测方法——CSPG。该方法分别从代码序列中提取语义特征，从代码属性图中提取图结构特征并使用MAFN对特征进行充分融合，在漏洞检测阶段通过引入插值预测分类器，有效提升模型的泛化性并优化漏洞检测效果。在3个典型数据集上的实验结果表明，CSPG具有良好漏洞检测效果。

本文提出的CSPG通过学习代码语义和结构特征进行漏洞检测，但尚未引入其他维度的代码特征，可能导致模型对代码的理解存在一定局限性。在未来研究中，将尝试结合更多有效代码特征以丰富模型学习内容，尝试结合利用大语言模型辅助进行漏洞检测，进一步提高漏洞检测性能。

参考文献

- [1] 苏小红, 郑伟宁, 蒋远, 等. 基于学习的源代码漏洞检测研究与进展[J]. 计算机学报, 2024, 47(2): 337–374. doi: [10.11897/SP.J.1016.2024.00337](https://doi.org/10.11897/SP.J.1016.2024.00337).
SU Xiaohong, ZHENG Weining, JIANG Yuan, et al. Research and progress on learning-based source code

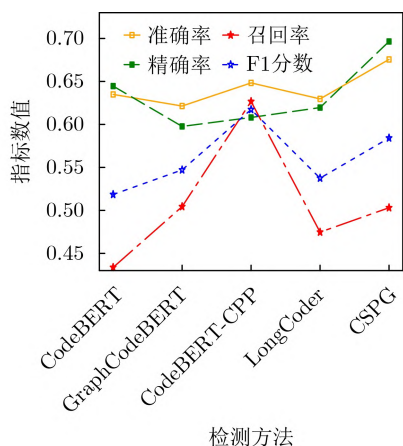


图5 各预训练模型组合性能对比

- vulnerability detection[J]. *Chinese Journal of Computers*, 2024, 47(2): 337–374. doi: [10.11897/SP.J.1016.2024.00337](https://doi.org/10.11897/SP.J.1016.2024.00337).
- [2] FU M and TANTITHAMTHAVORN C. LineVul: A transformer-based line-level vulnerability prediction[C]. The 19th International Conference on Mining Software Repositories, Pittsburgh, USA, 2022: 608–620. doi: [10.1145/3524842.3528452](https://doi.org/10.1145/3524842.3528452).
 - [3] LI Zhen, ZOU Deqing, XU Shouhuai, *et al.* SySeVR: A framework for using deep learning to detect software vulnerabilities[J]. *IEEE Transactions on Dependable and Secure Computing*, 2022, 19(4): 2244–2258. doi: [10.1109/TDSC.2021.3051525](https://doi.org/10.1109/TDSC.2021.3051525).
 - [4] XIA Yuying, SHAO Haijian, and DENG Xing. VulCoBERT: A CodeBERT-based system for source code vulnerability detection[C]. The 2024 International Conference on Generative Artificial Intelligence and Information Security, Guangzhou, China, 2024: 249–252. doi: [10.1145/3665348.3665391](https://doi.org/10.1145/3665348.3665391).
 - [5] DU Gewangzi, CHEN Liwei, WU Tongshuai, *et al.* CPMSVD: Cross-project multiclass software vulnerability detection via fused deep feature and domain adaptation[C]. The 49th IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Seoul, South Korea, 2024: 4950–4954. doi: [10.1109/ICASSP48485.2024.10447552](https://doi.org/10.1109/ICASSP48485.2024.10447552).
 - [6] SHESTOV A, LEVICHEV R, MUSSABAYEV R, *et al.* Finetuning large language models for vulnerability detection[J]. *IEEE Access*, 2025, 13: 38889–38900. doi: [10.1109/ACCESS.2025.3546700](https://doi.org/10.1109/ACCESS.2025.3546700).
 - [7] DO C X, LUU N T, and NGUYEN P T L. Optimizing software vulnerability detection using RoBERTa and machine learning[J]. *Automated Software Engineering*, 2024, 31(2): 40. doi: [10.1007/s10515-024-00440-1](https://doi.org/10.1007/s10515-024-00440-1).
 - [8] FENG Zhangyin, GUO Daya, TANG Duyu, *et al.* CodeBERT: A pre-trained model for programming and natural languages[C]. Findings of the Association for Computational Linguistics: EMNLP 2020, 2020: 1536–1547. doi: [10.18653/v1/2020.findings-emnlp.139](https://doi.org/10.18653/v1/2020.findings-emnlp.139).
 - [9] ZHOU Yaqin, LIU Shangqing, SLOW J, *et al.* Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks[C]. The 33rd International Conference on Neural Information Processing Systems, Vancouver, Canada, 2019: 915.
 - [10] CHAKRABORTY S, KRISHNA R, DING Yangruibo, *et al.* Deep learning based vulnerability detection: Are we there yet?[J]. *IEEE Transactions on Software Engineering*, 2022, 48(9): 3280–3296. doi: [10.1109/TSE.2021.3087402](https://doi.org/10.1109/TSE.2021.3087402).
 - [11] WEN Xincheng, GAO Cuiyun, GAO Shuzheng, *et al.* SCALE: Constructing structured natural language comment trees for software vulnerability detection[C]. The 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, Vienna, Austria, 2024: 235–247. doi: [10.1145/3650212.3652124](https://doi.org/10.1145/3650212.3652124).
 - [12] LIU Ruitong, WANG Yanbin, XU Haitao, *et al.* Vul-LMGNNs: Fusing language models and online-distilled graph neural networks for code vulnerability detection[J]. *Information Fusion*, 2025, 115: 102748. doi: [10.1016/j.inffus.2024.102748](https://doi.org/10.1016/j.inffus.2024.102748).
 - [13] TANG Mingwei, TANG Wei, GUI Qingchi, *et al.* A vulnerability detection algorithm based on Residual Graph Attention Networks for source code imbalance (RGAN)[J]. *Expert Systems with Applications*, 2024, 238: 122216. doi: [10.1016/j.eswa.2023.122216](https://doi.org/10.1016/j.eswa.2023.122216).
 - [14] SHAO Miaomiao, DING Yuxin, CAO Jing, *et al.* GraphFVD: Property graph-based fine-grained vulnerability detection[J]. *Computers & Security*, 2025, 151: 104350. doi: [10.1016/j.cose.2025.104350](https://doi.org/10.1016/j.cose.2025.104350).
 - [15] 胡雨涛, 王溯远, 吴月明, 等. 基于图神经网络的切片级漏洞检测及解释方法[J]. *软件学报*, 2023, 34(6): 2543–2561. doi: [10.13328/j.cnki.jos.006849](https://doi.org/10.13328/j.cnki.jos.006849).
 - HU Yutao, WANG Suyuan, WU Yueming, *et al.* Slice-level vulnerability detection and interpretation method based on graph neural network[J]. *Journal of Software*, 2023, 34(6): 2543–2561. doi: [10.13328/j.cnki.jos.006849](https://doi.org/10.13328/j.cnki.jos.006849).
 - [16] QIU Fangcheng, LIU Zhongxin, HU Xing, *et al.* Vulnerability detection via multiple-graph-based code representation[J]. *IEEE Transactions on Software Engineering*, 2024, 50(8): 2178–2199. doi: [10.1109/tse.2024.3427815](https://doi.org/10.1109/tse.2024.3427815).
 - [17] ZHANG Guodong, YAO Tianyu, QIN Jiawei, *et al.* CodeSAGE: A multi-feature fusion vulnerability detection approach using code attribute graphs and attention mechanisms[J]. *Journal of Information Security and Applications*, 2025, 89: 103973. doi: [10.1016/j.jisa.2025.103973](https://doi.org/10.1016/j.jisa.2025.103973).
 - [18] GUO Daya, LU Shuai, DUAN Nan, *et al.* UniXcoder: Unified cross-modal pre-training for code representation[C]. The 60th Annual Meeting of the Association for Computational Linguistics, Dublin, Ireland, 2022: 7212–7225. doi: [10.18653/v1/2022.acl-long.499](https://doi.org/10.18653/v1/2022.acl-long.499).
 - [19] BRESSON X and LAURENT T. Residual gated graph convnets[EB/OL]. <https://arxiv.org/abs/1711.07553>, 2017.
 - [20] LIN Yuxiao, MENG Yuxian, SUN Xiaofei, *et al.* BertGCN: Transductive text classification by combining GNN and BERT[C]. Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021, Bangkok, Thailand, 2021: 1456–1462. doi: [10.18653/v1/2021.findings-acl.126](https://doi.org/10.18653/v1/2021.findings-acl.126).

- [21] NI Chao, YIN Xin, YANG Kaiwen, *et al.* Distinguishing look-alike innocent and vulnerable code by subtle semantic representation learning and explanation[C]. The 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, San Francisco, USA, 2023: 1611–1622. doi: [10.1145/3611643.3616358](https://doi.org/10.1145/3611643.3616358).
- [22] GUO Daya, REN Shuo, LU Shuai, *et al.* GraphCodeBERT: Pre-training code representations with data flow[C]. 9th International Conference on Learning Representations, 2021.
- [23] ZHOU Shuyan, ALON U, AGARWAL S, *et al.* CodeBERTScore: Evaluating code generation with pretrained models of code[C]. The 2023 Conference on Empirical Methods in Natural Language Processing, Singapore, 2023: 13921–13937. doi: [10.18653/v1/2023.emnlp-main.859](https://doi.org/10.18653/v1/2023.emnlp-main.859).
- [24] GUO Daya, XU Canwen, DUAN Nan, *et al.* LongCoder: A long-range pre-trained language model for code completion[C]. The 40th International Conference on Machine Learning, Honolulu, USA, 2023: 12098–12107.
- 杨宏宇: 男, 教授, 博士生导师, 研究方向为网络与系统安全、软件安全、网络安全态势感知。
- 罗靖川: 男, 硕士生, 研究方向为网络与信息安全。
- 成 翔: 男, 讲师, 研究方向为网络与系统安全、网络安全态势感知、APT攻击检测。
- 胡俊成: 男, 讲师, 研究方向为计算机体系结构、物联网大数据、人工智能与安全。
- 责任编辑: 马秀强

Source Code Vulnerability Detection Method Integrating Code Sequences and Property Graphs

YANG Hongyu^{①②} LUO Jingchuan^① CHENG Xiang^③ HU Juncheng^④

^①(School of Safety Science and Engineering, Civil Aviation University of China, Tianjin 300300, China)

^②(School of Computer Science and Technology, Civil Aviation University of China, Tianjin 300300, China)

^③(School of Information Engineering, Yangzhou University, Yangzhou 225127, China)

^④(School of Computer Science and Technology, Jilin University, Changchun 130012, China)

Abstract:

Objective Code vulnerabilities create opportunities for hacker intrusions, and if they are not promptly identified and remedied, they pose serious threats to cybersecurity. Deep learning-based vulnerability detection methods leverage large collections of source code to learn secure programming patterns and vulnerability characteristics, enabling the automated identification of potential security risks and enhancing code security. However, most existing deep learning approaches rely on a single network architecture, extracting features from only one perspective, which constrains their ability to comprehensively capture multi-dimensional code characteristics. Some studies have attempted to address this by extracting features from multiple dimensions, yet the adopted feature fusion strategies are relatively simplistic, typically limited to feature concatenation or weighted combination. Such strategies fail to capture interdependencies among feature dimensions, thereby reducing the effectiveness of feature fusion. To address these challenges, this study proposes a source code vulnerability detection method integrating code sequences and property graphs. By optimizing both feature fusion and vulnerability detection processes, the proposed method effectively enhances the accuracy and robustness of vulnerability detection.

Methods The proposed method consists of four components: feature representation, feature extraction, feature fusion, and vulnerability detection (Fig. 1). First, vector representations of the code sequence and the Code Property Graph (CPG) are obtained. Using word embedding and node embedding techniques, the code sequence and graph nodes are mapped into fixed-dimensional vectors, which serve as inputs for subsequent feature extraction. Next, the pre-trained UniXcoder model is employed to capture contextual information and extract semantic features from the code. In parallel, a Residual Gated Graph Convolution Network (RGGCN) is applied to the CPG to capture complex structural information, thereby extracting graph structural features. To integrate these complementary representations, a Multimodal Attention Fusion Network (MAFN) is designed to model the interactions between semantic and structural features. This network generates

informative fused features for the vulnerability detection task. Finally, a MultiLayer Perceptron (MLP) performs classification on the semantic features, structural features, and fused features. An interpolated prediction classifier is then applied to optimize the detection process by balancing multiple prediction outcomes. By adaptively adjusting the model's focus according to the characteristics of different code samples, the classifier enables the detection model to concentrate on the most critical features, thereby improving overall detection accuracy.

Results and Discussions To validate the effectiveness of the proposed method, comparative experiments were conducted against baseline approaches on the Devign, Reveal, and SVulD datasets. The experimental results are summarized in (Tables 1–3). On the Devign dataset, the proposed method achieved an accuracy improvement of 1.38% over SCALE and a precision improvement of 5.19% over CodeBERT. On the Reveal dataset, it improved accuracy by 0.08% compared to SCALE, with precision being closest to that of SCALE. On the SVulD dataset, the method achieved an accuracy improvement of 0.13% over SCALE and a precision gain of 8.15% over Vul-LMGNNs. Collectively, these results demonstrate that the proposed method consistently yields higher accuracy and precision. This improvement can be attributed to its effective integration of semantic information extracted by UniXcoder and structural information captured by RGGCN. By contrast, CodeBERT and LineVul effectively learn code semantics but exhibit insufficient understanding of complex structural patterns, resulting in weaker detection performance. Devign and Reveal employ gated graph neural networks to capture structural information from code graphs but lack the ability to model semantic information contained in code sequences, which constrains their performance. Vul-LMGNNs attempt to improve detection performance by jointly learning semantic and structural features; however, their feature fusion strategy relies on simple concatenation. This approach fails to account for correlations between features, severely limiting the expressive power of the fused representation and reducing detection performance. In contrast, the proposed method fully leverages and integrates semantic and structural features through multimodal attention fusion. By modeling feature interactions rather than treating them independently, it achieves superior accuracy and precision, enabling more effective vulnerability detection.

Conclusions Fully integrating code features across multiple dimensions can significantly enhance vulnerability detection performance. Compared with baseline methods, the proposed approach enables deeper modeling of interactions among code features, allowing the detection model to develop a more comprehensive understanding of code characteristics and thereby achieve superior detection accuracy and precision.

Key words: Vulnerability detection; Code Property Graph (CPG); Pre-trained model; Interpolation prediction